WEB APPLICATION SECURITY AUDIT REPORT
of
BCAS | Ministry of Electronics & Information Technology, Government of India WEBSITE: https://uat.nielit.gov.in/bcas/

Submitted By
Secnic Consultancy

Version 4.0
May 2026

BCAS | Ministry of Electronics & Information Technology, Government of India.

All rights reserved. All information contained in this document is confidential and proprietary to BCAS | Ministry of Electronics & Information Technology, Government of India No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner without the prior written consent of BCAS | Ministry of Electronics & Information Technology, Government of India.

All brands or product names are trademarks or registered trademarks of their respective companies or organizations.

TABLE OF CONTENTS

1.1 INTRODUCTION 3

1.2 SCOPE 3

1.3 SUMMARY OF FINDINGS 4

1.4 OVERALL ASSESSMENT 4

1.5 KEY VULNERABILITIES DISCOVERED 5

1. EXECUTIVE SUMMARY

1.1 INTRODUCTION

This report presents the results of the penetration testing for the BCAS | Ministry of Electronics & Information Technology , Government of India Website (https://uat.nielit.gov.in/bcas/). The objective of the test was to find out vulnerabilities that can be seen and compromised by malicious users from the Internet. The application was found to be vulnerable to security risks including Security Misconfiguration, & Denial of Service issues etc... The following sections provide details of the scope of the penetration test, the IPs on which detailed penetration testing was conducted, and the detailed findings from the penetration testing exercise.

Our opinion provided in this report is valid for the period during which the penetration testing was carried out and is based on the information and the website contents provided for audit. Projection of any conclusions based on our findings for future periods and application versions is subject to the risk that the validity of such conclusions may be altered because of changes made to the network or application or system or the failure to make the changes to the network, application, system when required.

1.2 SCOPE

The following table details an overview on the scope of the penetration test.

Details	
Application Name	**BCAS | Ministry of Electronics & Information Technology , Government of India**
Testing IP/URL	**https://uat.nielit.gov.in/bcas/**
Mode of Test	**Gray Box**

1.3 SUMMARY OF FINDINGS

Penetration testing on the Website (https://uat.nielit.gov.in/bcas/) was performed for a period of 15 days. As a result of the penetration test following risks were identified:

No of Findings	*Critical*	*High*	*Medium*	*Low*
02	**0**	**2**	**0**	**0**

The table below gives an overall result of the findings from the penetration testing exercise. The "Key Findings and Action Items" section gives the details of the vulnerabilities and provides the recommendations to mitigate them.

1.4 OVERALL ASSESSMENT

We have identified 2 High, and 0 Medium, and 0 Low vulnerabilities during the penetration testing of the (https://uat.nielit.gov.in/bcas/). The team concluded the risks based on the results of the various tests carried out during the penetration testing process. The following table lists down the key vulnerabilities discovered.

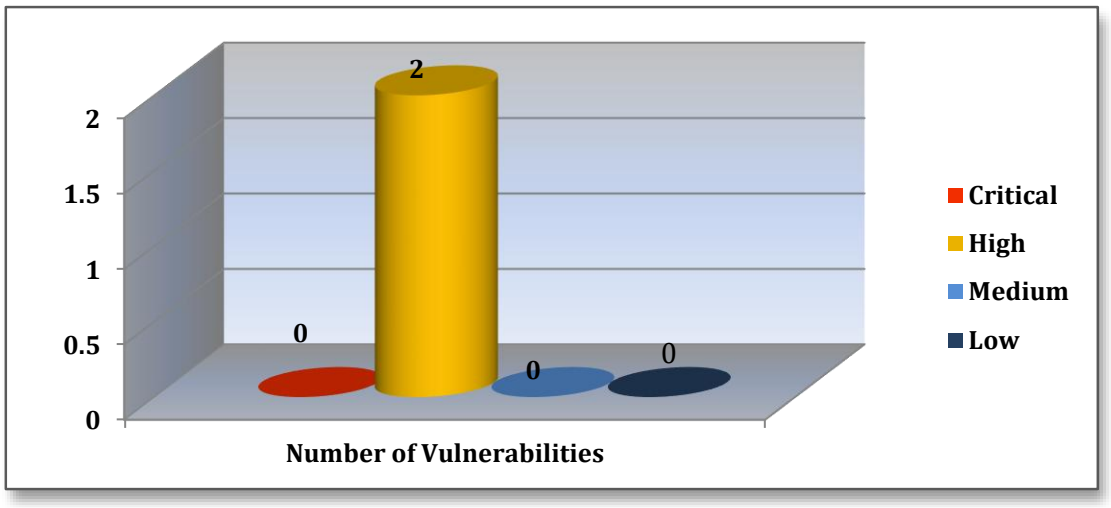
1.5 KEY VULNERABILITIES DISCOVERED

Sl. No.	VULNERABILITY	RISK LEVEL	ACTION ITEM	Phase 4 Status
1	No Rate Limiting	High	Implement rate limiting on authentication and sensitive endpoints. Apply IP-based and user-based thresholds. Introduce CAPTCHA after multiple failed attempts. Monitor abnormal traffic and block abusive IPs.	CLOSED
2	Cleartext Submission of Password	High	Enforce HTTPS across all endpoints. Implement strong TLS configurations and HSTS. Avoid transmitting passwords in plain text.	OPEN
3	Missing Content Security Policy (CSP) Header	High	Implement strong CSP with restricted sources. Avoid inline scripts. Use nonce/hash-based CSP. Regularly review and enforce policy.	OPEN
4	Missing HTTP Strict Transport Security (HSTS)	High	Configure HSTS with max-age and includeSubDomains. Enforce HTTPS across application. Add to preload list if applicable.	CLOSED
5	TLS 1.3 Not Enabled / Protocol Downgrade Risk	Medium	Enable and enforce TLS 1.3. Disable protocol downgrade fallback. Use strong cipher suites. Regularly update SSL/TLS configurations.	RISK ACCEPTANCE

6	Missing Referrer-Policy Header	Low	Implement strict Referrer-Policy such as no-referrer or strict-origin-when-cross-origin. Avoid sensitive data in URLs.	**CLOSED**
7	Missing Permissions-Policy Header	Low	Implement Permissions-Policy to restrict unnecessary browser features. Disable sensitive APIs unless required. Follow least privilege principle.	**CLOSED**
8	Deprecated Security Header – X-XSS-Protection	Low	Remove deprecated header. Implement strong CSP. Apply input validation and output encoding to prevent XSS.	**CLOSED**
9	Use of Outdated JavaScript Library (jQuery UI 1.12.1)	Low	Upgrade to latest version. Remove unused libraries. Monitor dependencies regularly. Perform periodic security audits.	**CLOSED**

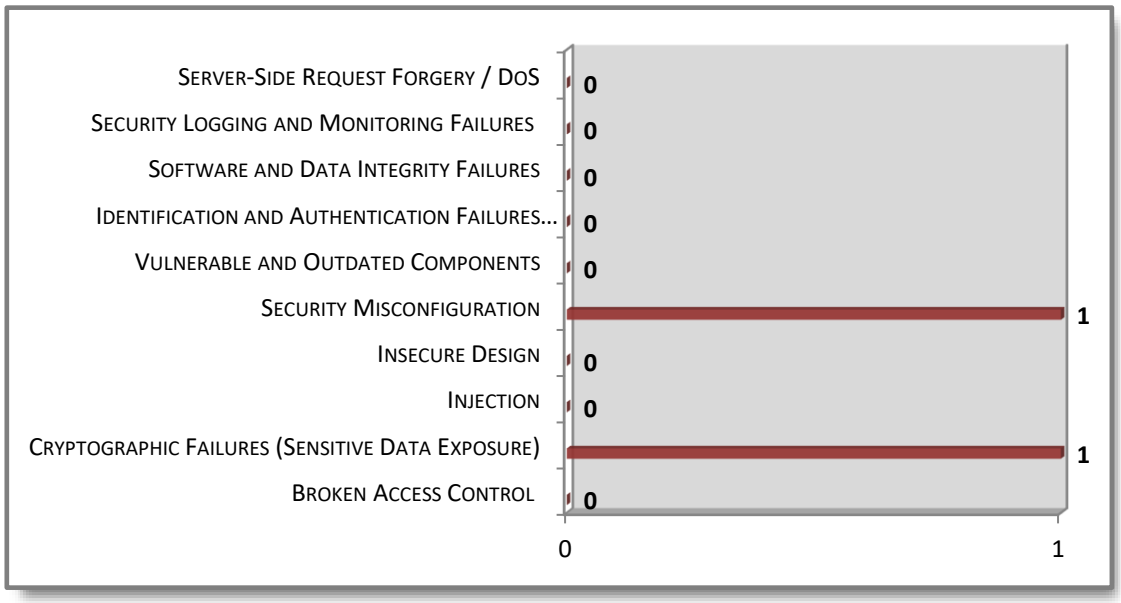
2. OVERALL SUMMARY OF FINDINGS

Below Chart gives the overall summary of number of Vulnerabilities discovered with the Risk rating. Total of 02 Vulnerabilities were reported.



3. BENCHMARKING WITH OWASP

The below chart depicts the number of discovered vulnerabilities mapped to OWASP Testing Category.



OWASP has rated the Top Ten Vulnerabilities found in web applications worldwide. OWASP Top Ten Vulnerabilities details can be found at below link.

https://owasp.org/Top10/

The table shows the application vulnerability mapped with respect to the OWASP Top 10 - 2021 list.

SL. NO	VULNERABILITIES	STATUS
1	Broken Access Control	PROTECTED
2	Cryptographic Failures (Sensitive Data Exposure)	UNPROTECTED
3	Injection	PROTECTED
4	Insecure Design	PROTECTED
5	Security Misconfiguration	UNPROTECTED
6	Vulnerable and Outdated Components	PROTECTED
7	Identification and Authentication Failures (Broken Authentication)	PROTECTED
8	Software and Data Integrity Failures	PROTECTED
9	Security Logging and Monitoring Failures	PROTECTED
10	Server-Side Request Forgery / DoS	PROTECTED

4. KEY FINDINGS AND ACTION ITEMS

1. No rate limit

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The application does not restrict the number of requests a user or IP can send within a specific time period. This allows attackers to perform brute force, credential stuffing, and denial-of-service style attacks without any restriction.	**High**	**High**	**High**

Impact:

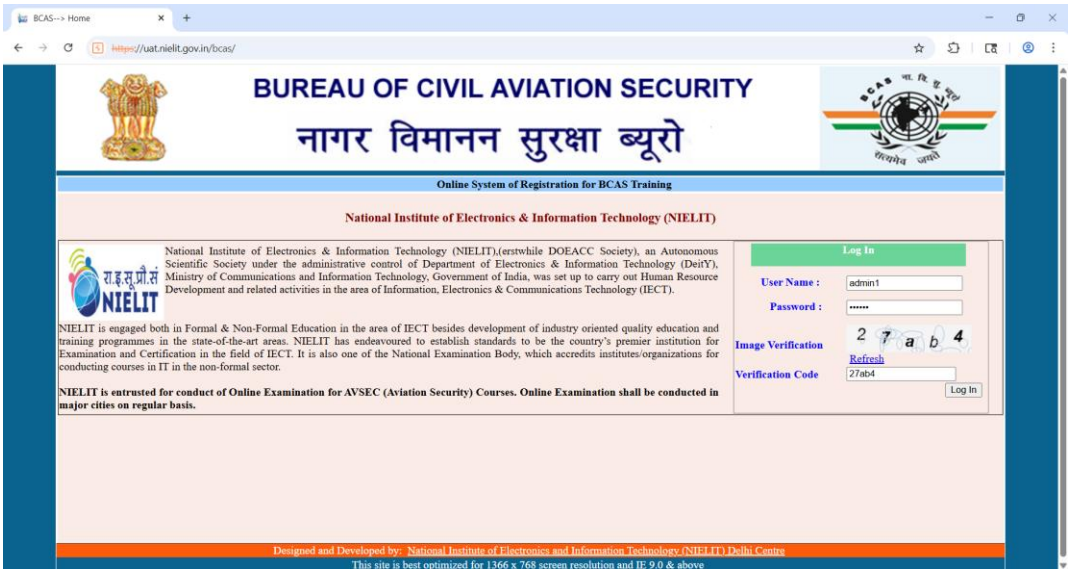
An attacker can brute-force user credentials, overload the application, bypass security controls, and compromise user accounts, leading to service disruption and data compromise.

Affected items:

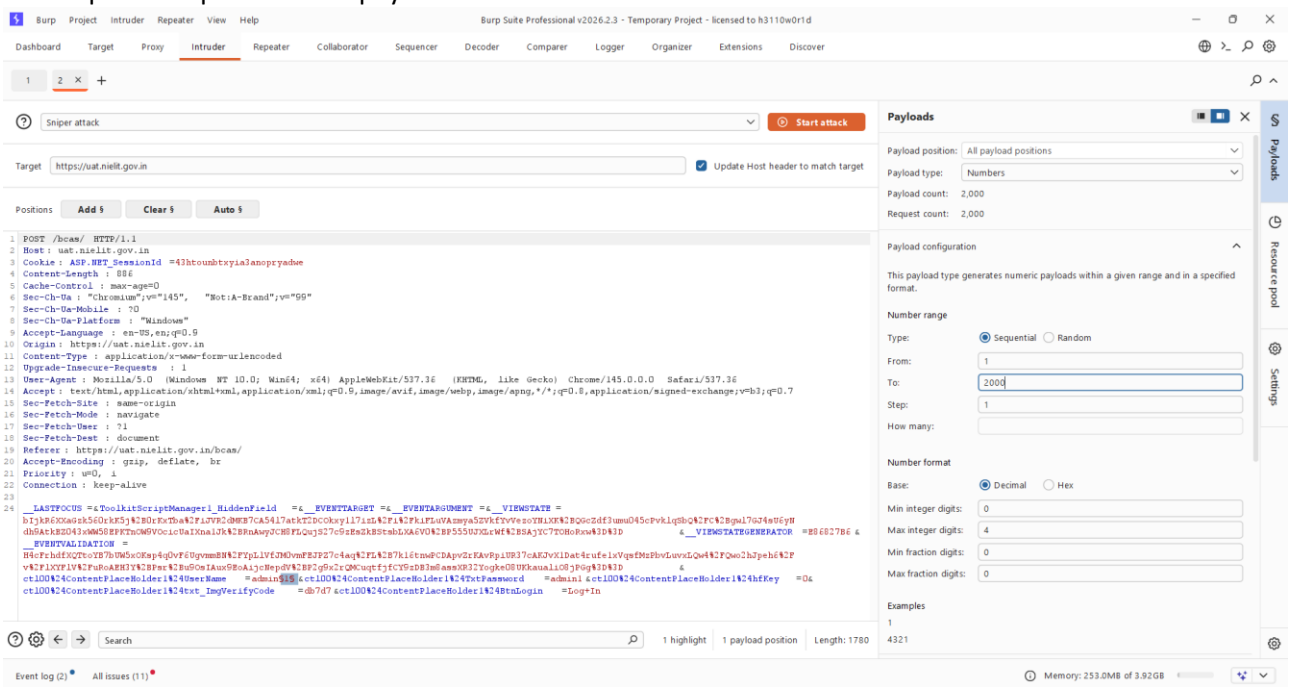
https://uat.nielit.gov.in/bcas/

Proof of Concept:

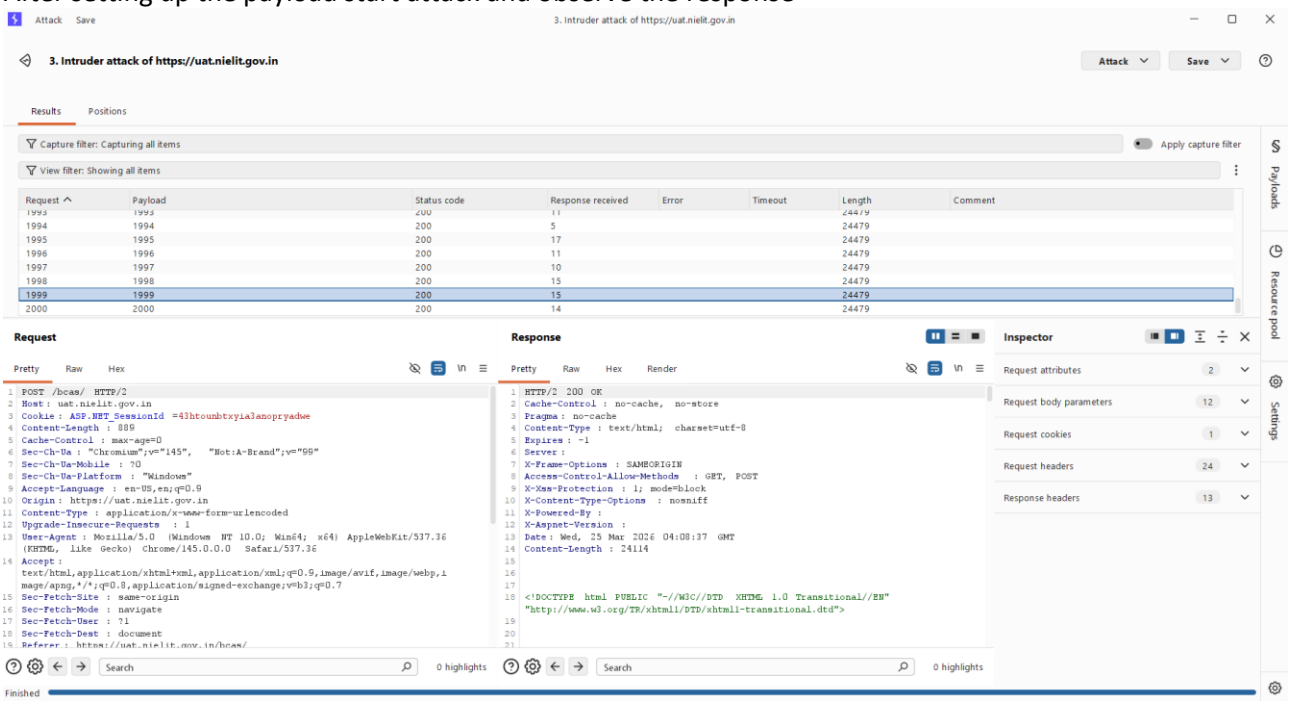
Enter any credentials and login



Intercept the request and set payload in intruder



After setting up the payload start attack and observe the response



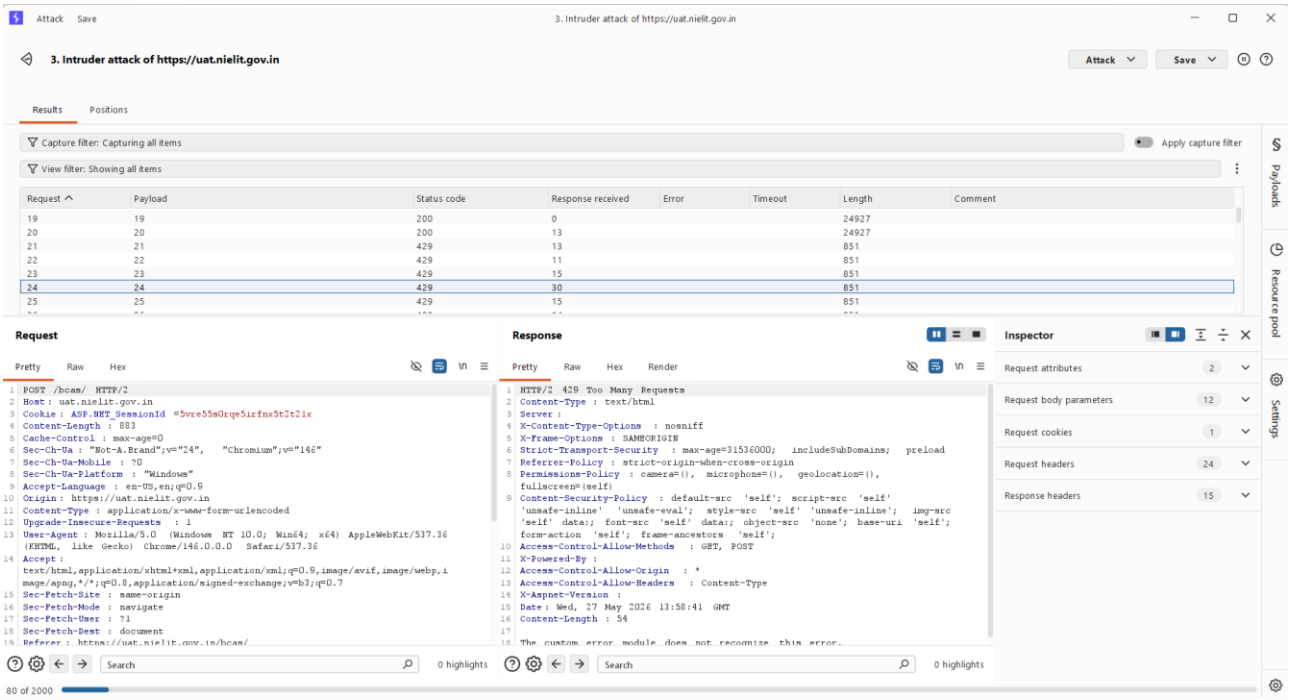
Recommendation:

- Implement rate limiting on authentication and sensitive endpoints.
- Apply IP-based and user-based request thresholds.
- Introduce CAPTCHA after multiple failed attempts.
- Log and monitor abnormal request patterns.
- Block or throttle abusive IP addresses.

References:

https://owasp.org/API-Security/editions/2019/en/0xa4-lack-of-resources-and-rate-limiting/

Phase 4:



2. Cleartext submission of password

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The application transmits user passwords in cleartext over the network without proper encryption protection. This allows attackers to intercept and read credentials using network sniffing or man-in-the-middle techniques.	**High**	**High**	**High**

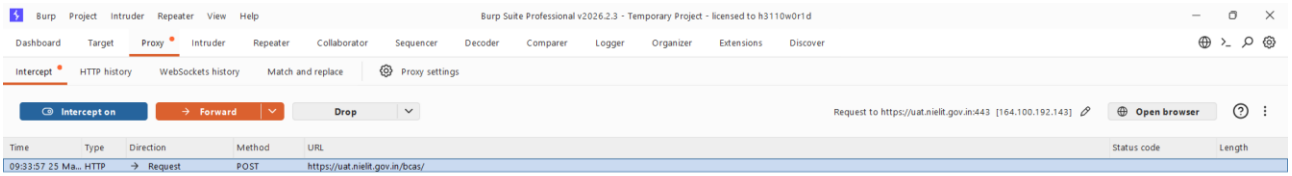
Impact:

An attacker can capture user credentials, gain unauthorized access to accounts, and reuse the same credentials on other platforms, leading to serious security and privacy risks.

Affected items:

- https://uat.nielit.gov.in/bcas//

Proof of Concept:





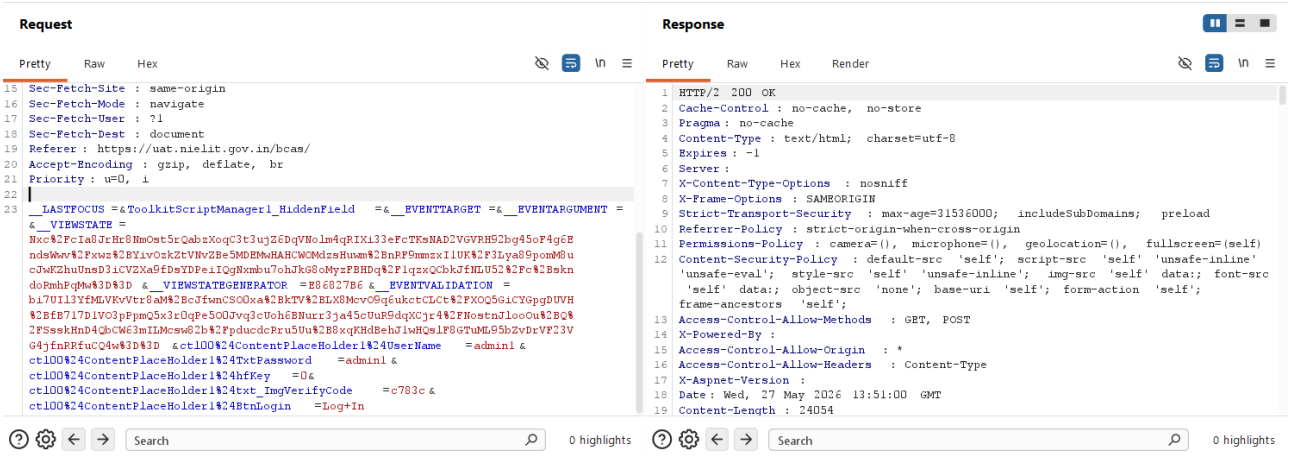
Recommendation:

- Enforce HTTPS for all authentication requests.
- Implement HSTS policy.
- Never transmit passwords in plain text.
- Use strong TLS configurations.

References:

https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html

Phase 4: *Ensure passwords are never transmitted in cleartext by enforcing HTTPS/TLS encryption with strong TLS configurations and HSTS across all endpoints.*



3. Missing Content Security Policy (CSP) Header

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The application does not implement a Content Security Policy (CSP) header in HTTP responses. This increases the risk of client-side attacks such as Cross-Site Scripting (XSS) and data injection.	**High**	**High**	**High**

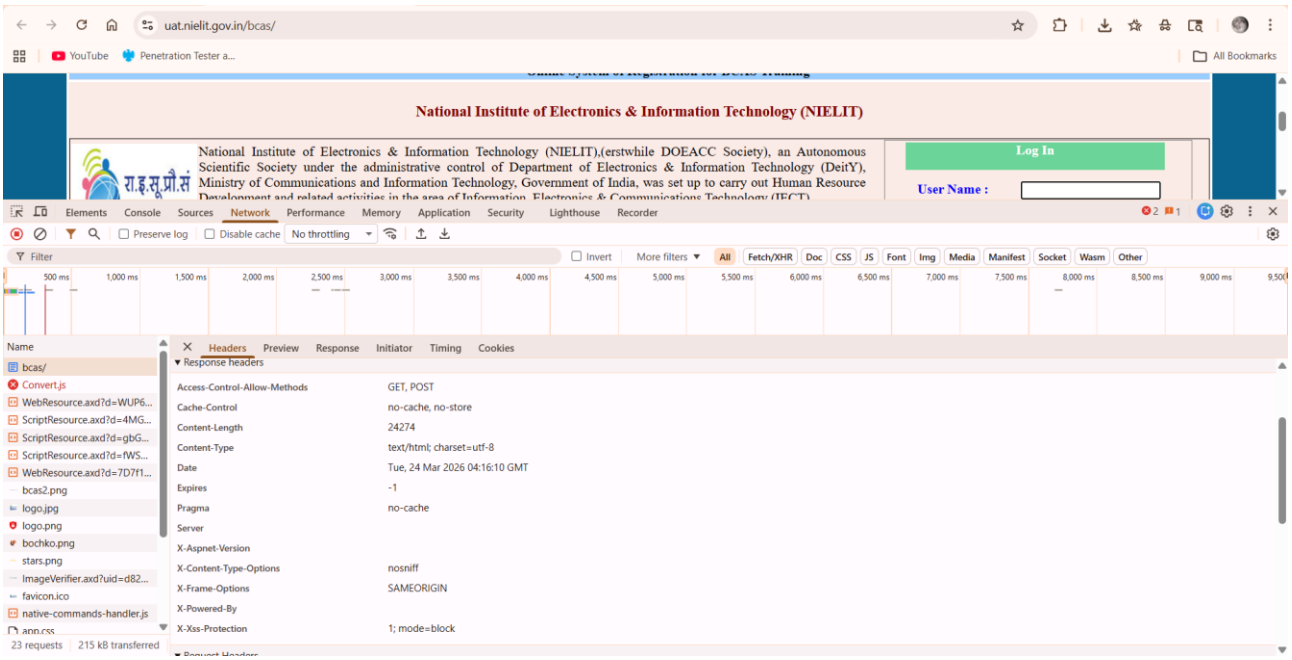
Impact:

An attacker can execute malicious scripts in the user's browser, steal session information, deface content, or redirect users to malicious websites.

Affected items:

https://uat.nielit.gov.in/bcas/

Proof of Concept:



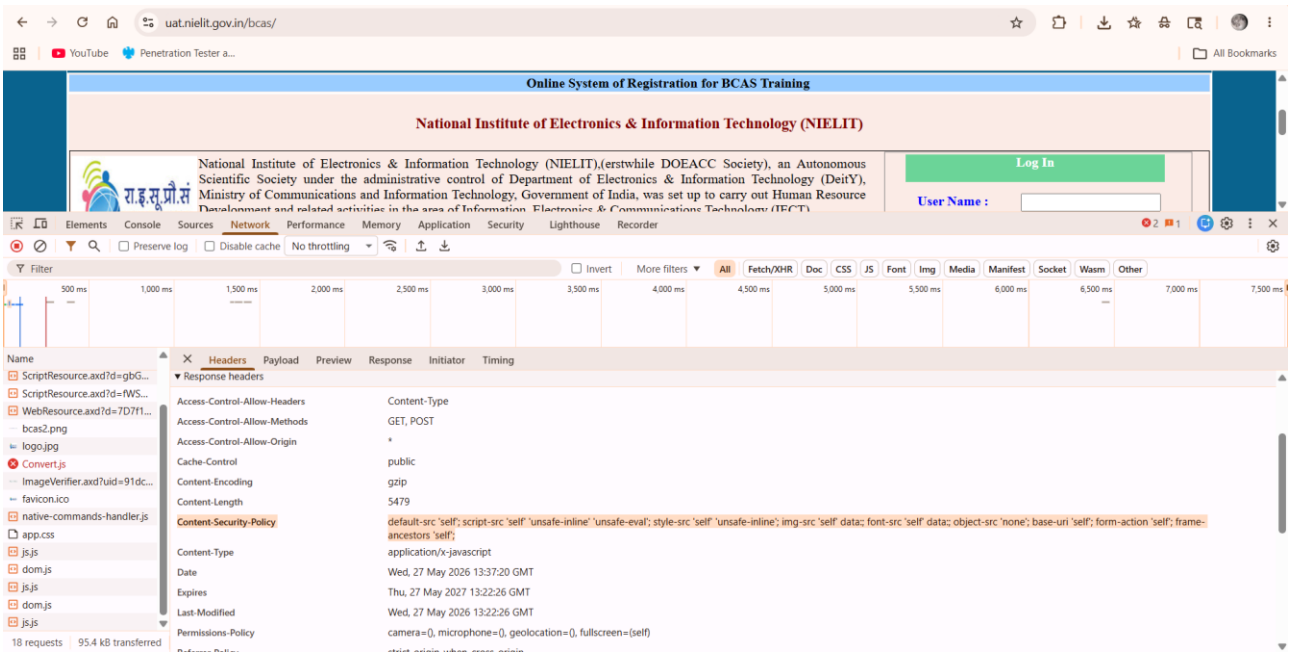
Recommendation:

- Implement a strong Content Security Policy header.
- Restrict script, style, and resource sources.
- Prevent inline script execution.
- Regularly review CSP rules.
- Combine CSP with proper input validation.

References:

https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

Phase 4: *Remove 'unsafe-inline' and 'unsafe-eval' and replace them with nonce- or hash-based CSP directives for strong compliance.*



4. Missing HTTP Strict Transport Security (HSTS)

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The application does not include the HTTP Strict Transport Security (HSTS) header in its responses. This allows browsers to access the application over insecure HTTP connections.	**Medium**	**Medium**	**Medium**

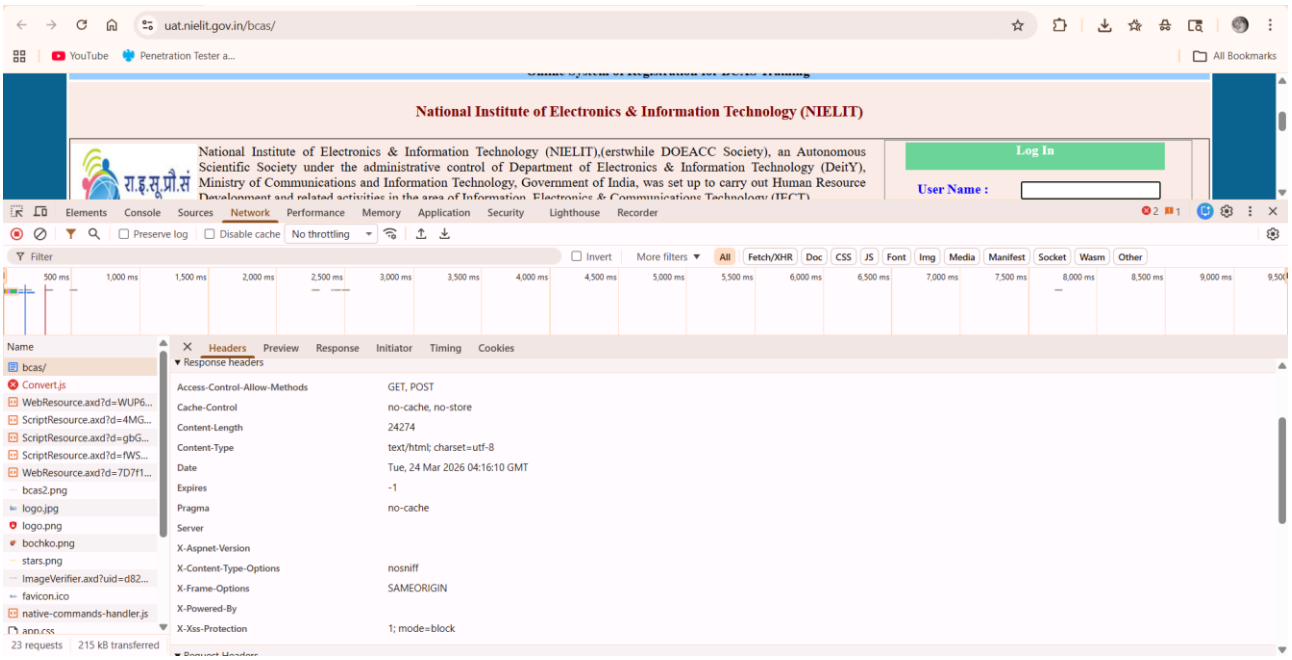
Impact:

An attacker can perform man-in-the-middle attacks, downgrade HTTPS connections, and intercept sensitive data such as login credentials and session cookies.

Affected items:

https://uat.nielit.gov.in/bcas/

Proof of Concept:



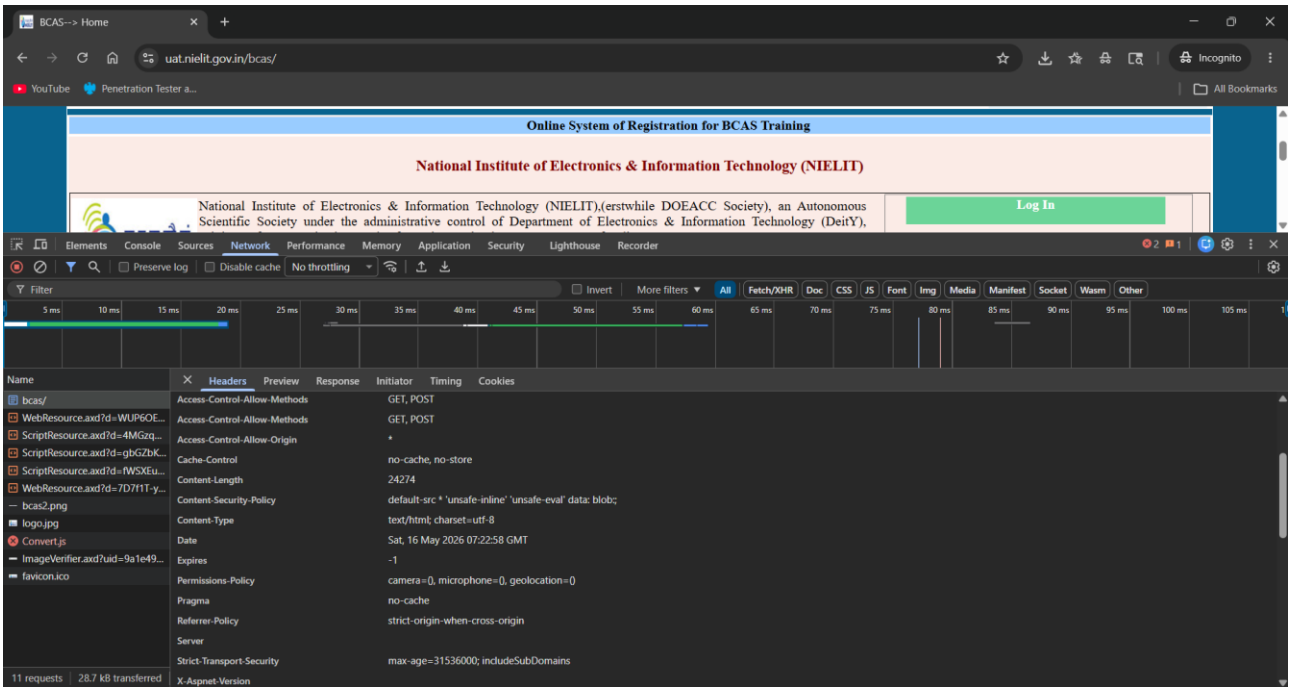
Recommendation:

- Enable HSTS with an appropriate max-age value.
- Include the includeSubDomains directive.
- Enforce HTTPS across the entire application.
- Add the domain to browser preload lists if applicable.
- Regularly audit transport security.

References:

https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html

Phase 2:



5. TLS 1.3 Not Enabled / Protocol Downgrade Risk

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The server currently supports TLS 1.2 but does not properly enable or enforce TLS 1.3, as observed during SSL testing. The absence of TLS 1.3 allows connections to fall back to older protocol versions, increasing the risk of protocol downgrade attacks. Although TLS 1.2 is still considered secure, it lacks the enhanced security and performance improvements provided by TLS 1.3.	**Medium**	**Medium**	**Low**

Impact:

- Susceptible to **protocol downgrade attacks**
- Reduced cryptographic strength compared to modern standards
- Increased attack surface for SSL/TLS exploitation
- Potential compliance gaps with modern security benchmarks

Affected items:

- https://uat.nielit.gov.in/bcas/

Proof of Concept:



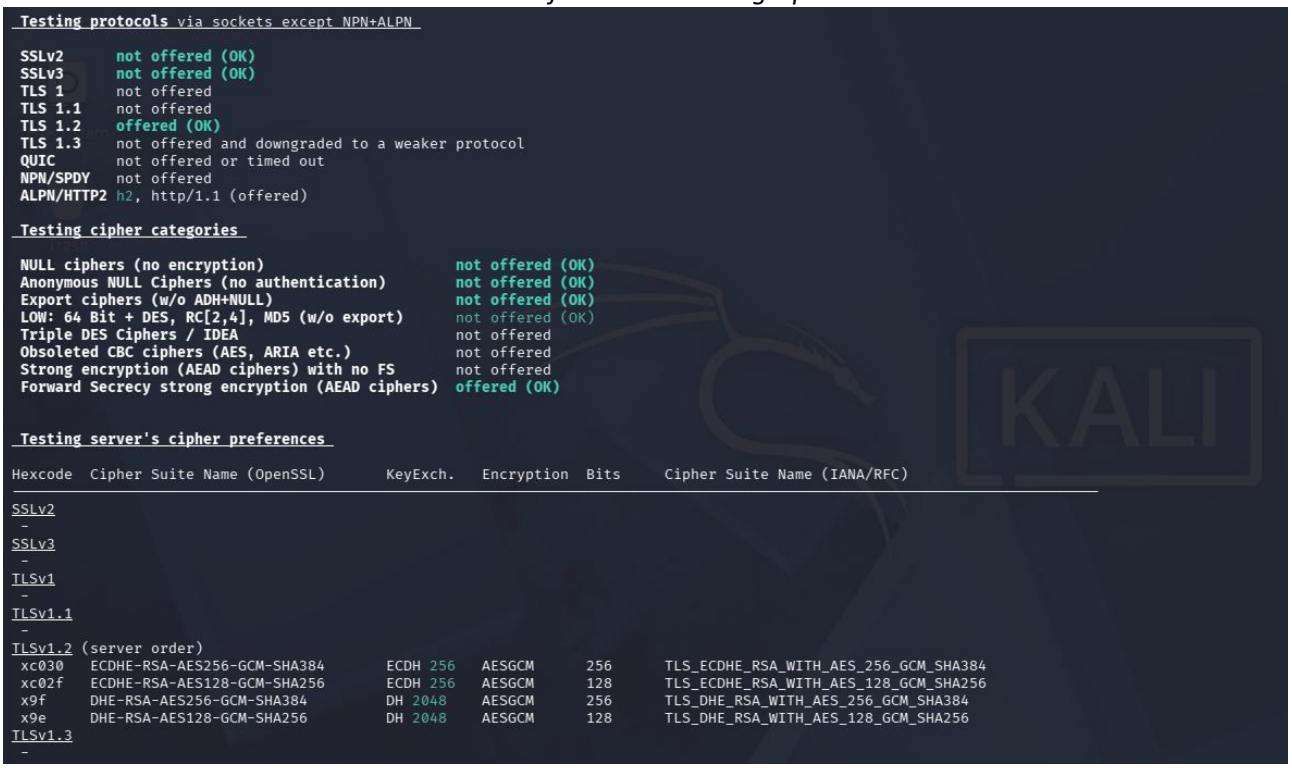
Recommendation:

- Enable and enforce **TLS 1.3** as the preferred protocol version.
- Disable fallback mechanisms that allow protocol downgrading.
- Configure server to prioritize **strong AEAD cipher suites with forward secrecy**.
- Update SSL/TLS libraries and server configurations regularly.
- Perform periodic SSL scans to ensure secure protocol enforcement.

References:

https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html

Phase 4:
Risk acceptance : *our server runs on Windows Server 2016 which does not support TLS 1.3 at the OS level. As a mitigation, TLS 1.0*
and 1.1 have been disabled and TLS 1.2 is enforced with strong cipher suites.



6. Missing Referrer-Policy Header

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*

The application does not include the Referrer-Policy HTTP header in its responses. This can cause the browser to send sensitive URL information (such as query parameters, tokens, or internal paths) to external domains via the Referer header.	**Low**	**Low**	**Low**

Impact:

An attacker can capture sensitive information through the Referer header when users navigate to external sites. This may expose:

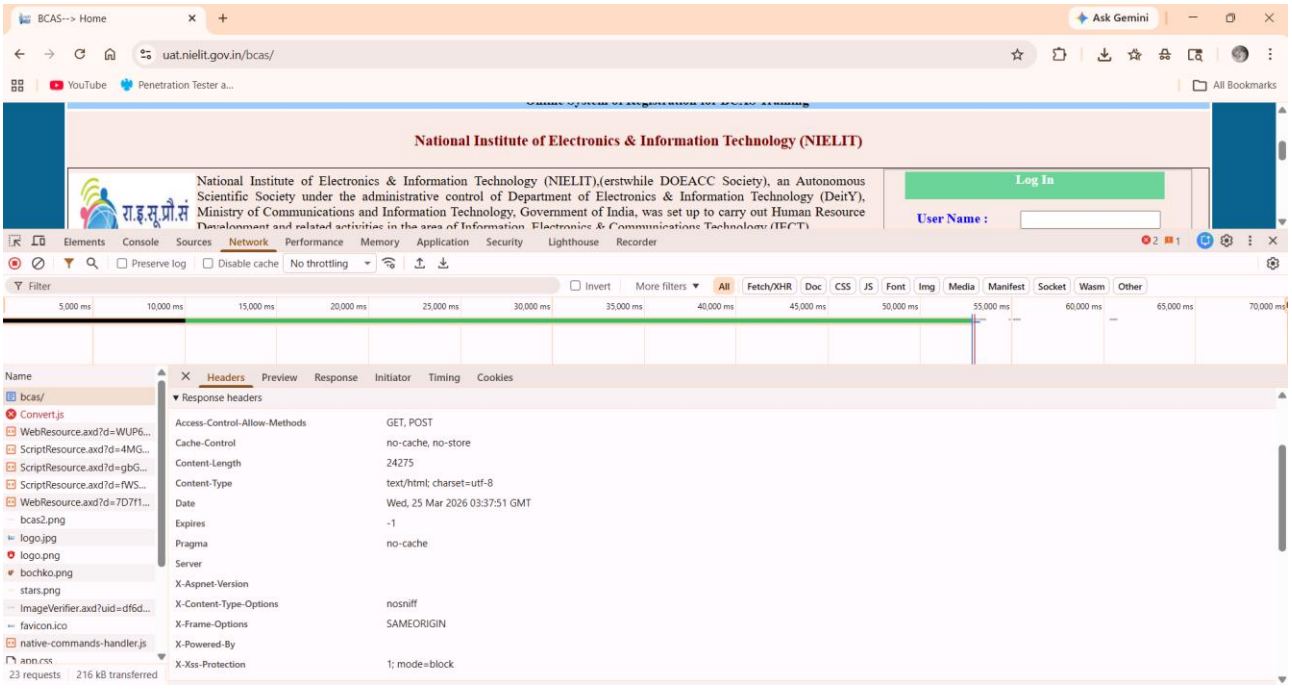
- Session tokens or sensitive query parameters
- Internal application paths and endpoints
- User activity and navigation patterns

This can lead to **information disclosure** and may assist in further attacks such as phishing or session hijacking.

Affected items:

- https://uat.nielit.gov.in/bcas/

Proof of Concept:



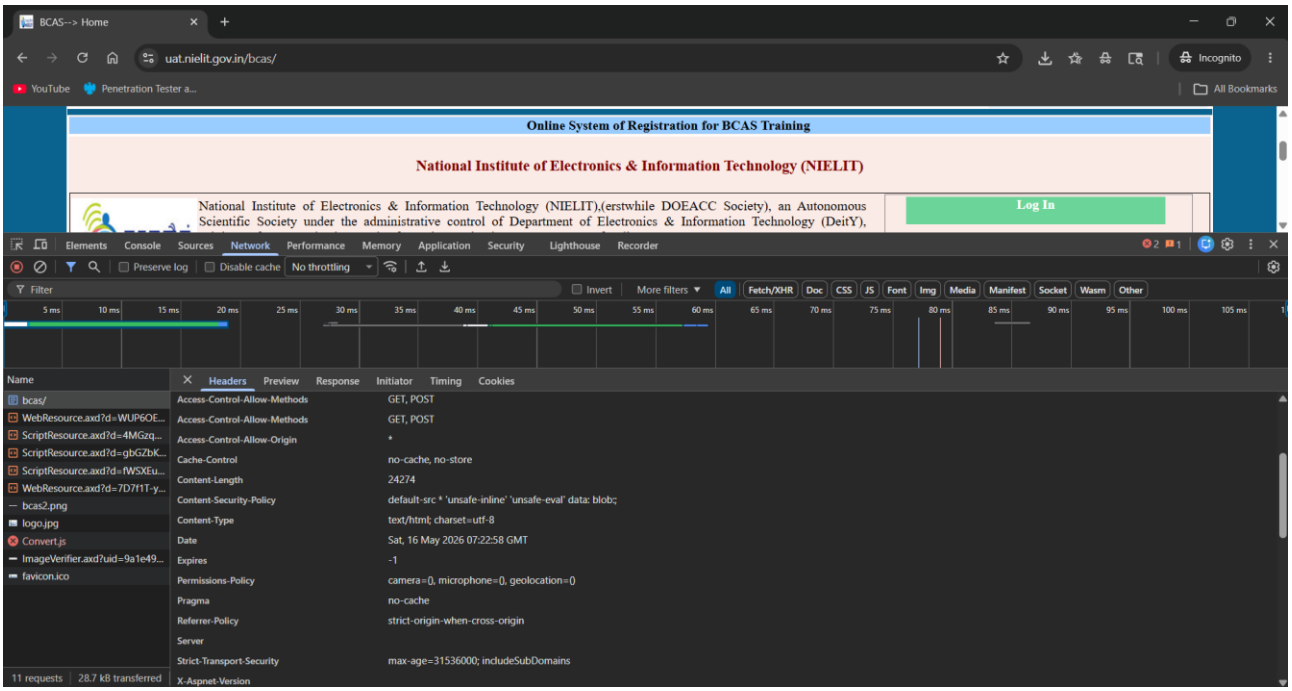
Recommendation:

- Implement Referrer-Policy header with a strict value like no-referrer or strict-origin-when-cross-origin.
- Avoid exposing sensitive data in URL parameters.
- Use HTTPS across the application to reduce leakage risk.
- Test application behavior after applying policy to ensure functionality.
- Regularly review headers as part of security audits.

References:

https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Referrer-Policy

Phase 2:



7. Missing Permissions-Policy Header

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The application does not include the Permissions-Policy HTTP header in its responses. This header controls access to powerful browser features such	**Low**	**Low**	**Low**

as camera, microphone, geolocation, fullscreen, and more. Without it, all features are allowed by default depending on browser behavior.			

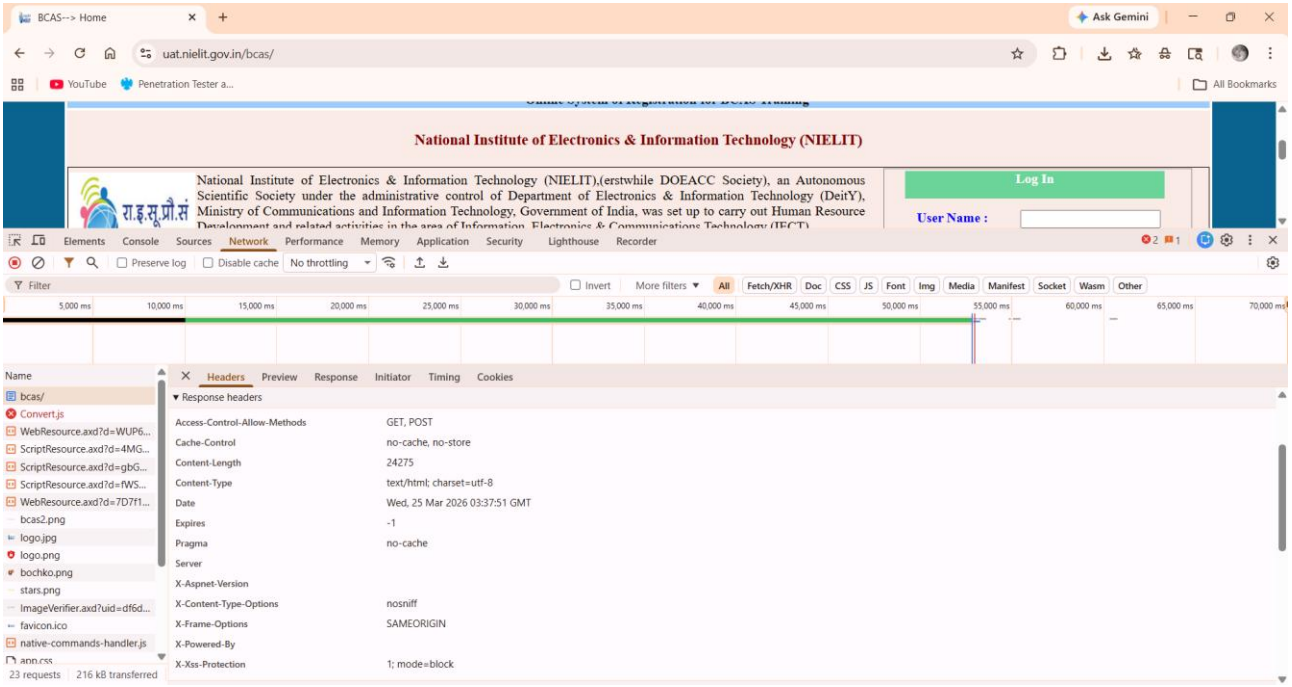
Impact:

- An attacker can abuse browser features if exploited through other vulnerabilities (e.g., XSS). This may lead to:
- Unauthorized access to camera or microphone
- Leakage of user location (geolocation API)
- Abuse of fullscreen or clipboard APIs
- Increased attack surface for client-side exploitation

Affected items:

- **https://uat.nielit.gov.in/bcas/**

Proof of Concept:



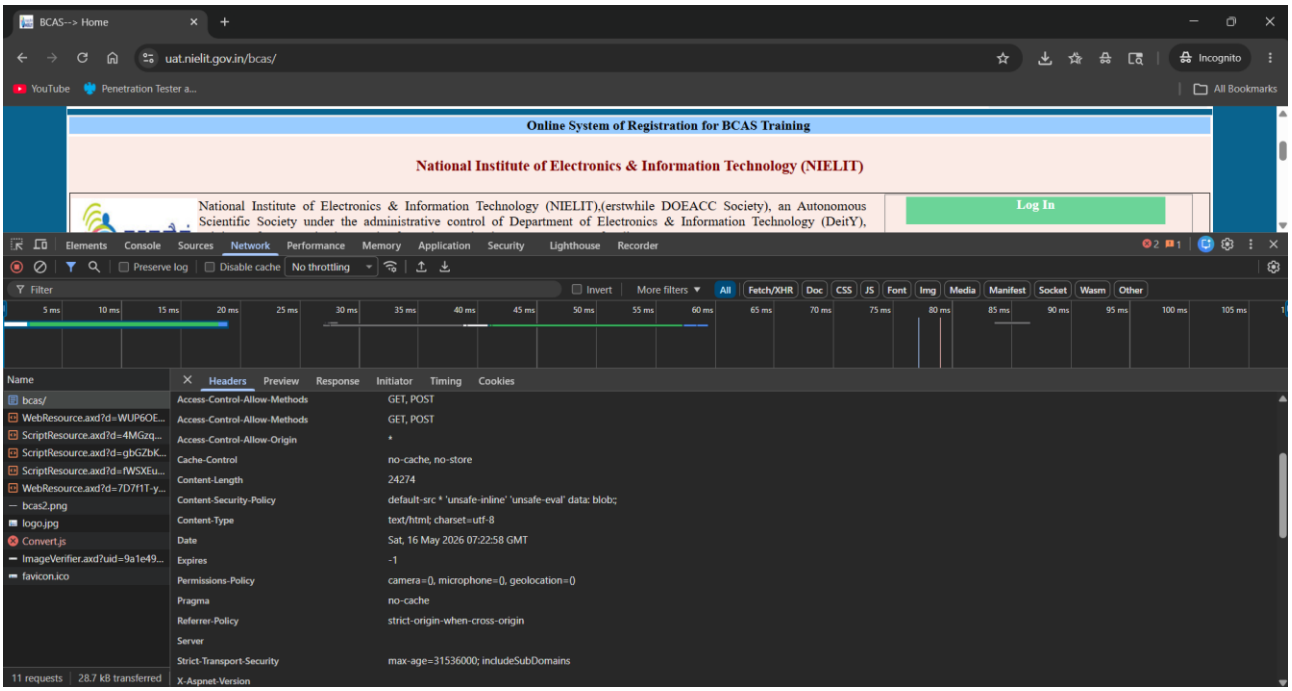
Recommendation:

- Implement Permissions-Policy header to restrict unnecessary browser features.
- Disable sensitive features like camera, microphone, and geolocation unless required.
- Use least privilege principle for browser capabilities.
- Test application functionality after applying restrictions.
- Regularly review and update policy based on feature usage.

References:

https://owasp.org/www-project-secure-headers/

Phase 2:



8. Deprecated Security Header – X-XSS-Protection

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The application uses the X-XSS-Protection header, which is deprecated and no longer supported by modern browsers. Relying on this header provides a false sense of security as it does not	**Low**	**Low**	**Low**

effectively prevent Cross-Site Scripting (XSS) attacks.			

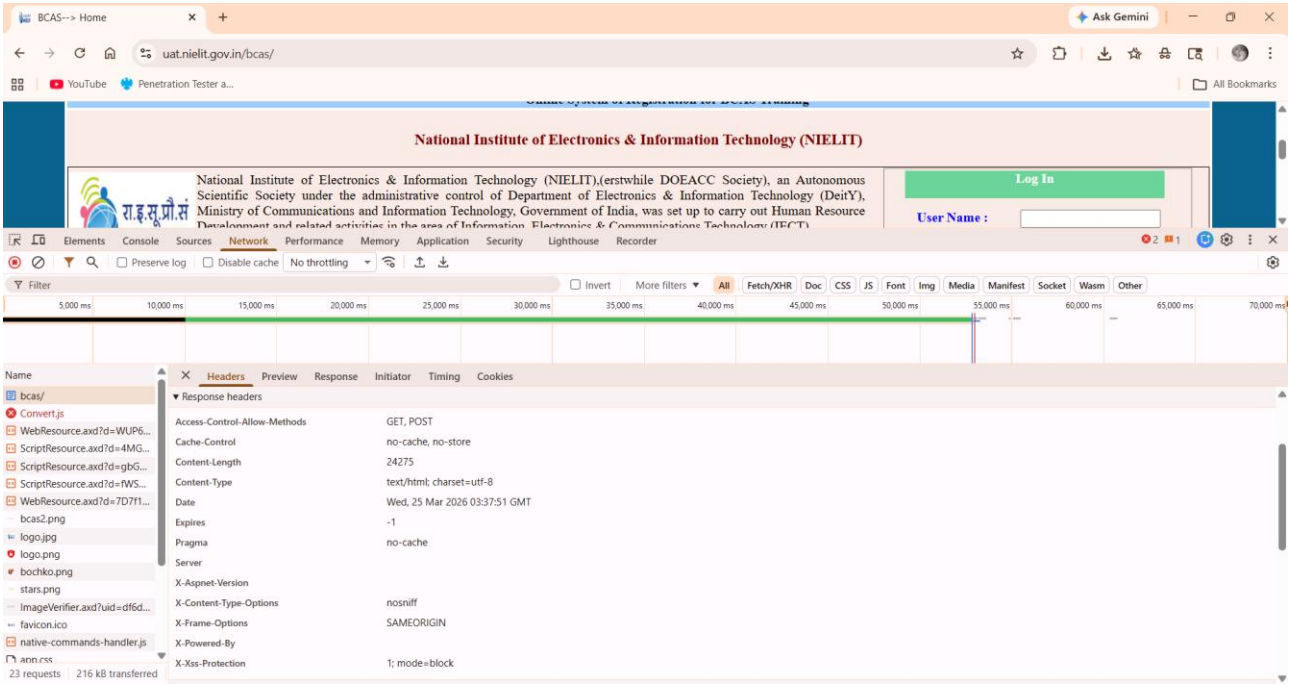
Impact:

Modern browsers ignore this header, so it does not provide any real protection against XSS attacks. This may lead to reliance on ineffective security controls and increased exposure to client-side attacks.

Affected items:

- https://uat.nielit.gov.in/bcas/

Proof of Concept:



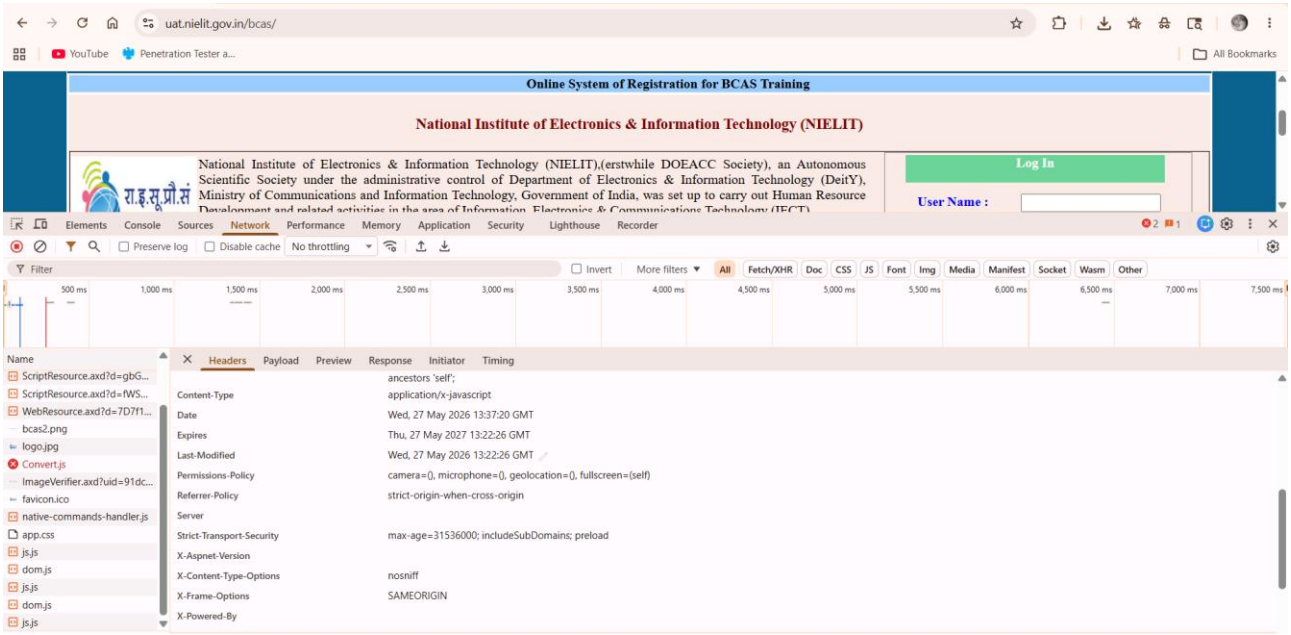
Recommendation:

- Remove the deprecated X-XSS-Protection header.
- Implement a strong Content Security Policy (CSP) instead.
- Validate and sanitize all user inputs on the server side.
- Use output encoding to prevent XSS vulnerabilities.
- Regularly test application for XSS flaws.

References:

https://owasp.org/www-project-secure-headers/

Phase 4:



9. Use of Outdated JavaScript Library (jQuery UI 1.12.1)

Brief Description	*Risk Rating*	*Impact on Application*	*Probability of Attack*
The application uses an outdated version of the jQuery UI library (1.12.1), which is no longer actively maintained and may contain known security vulnerabilities.	**Low**	**Low**	**Low**

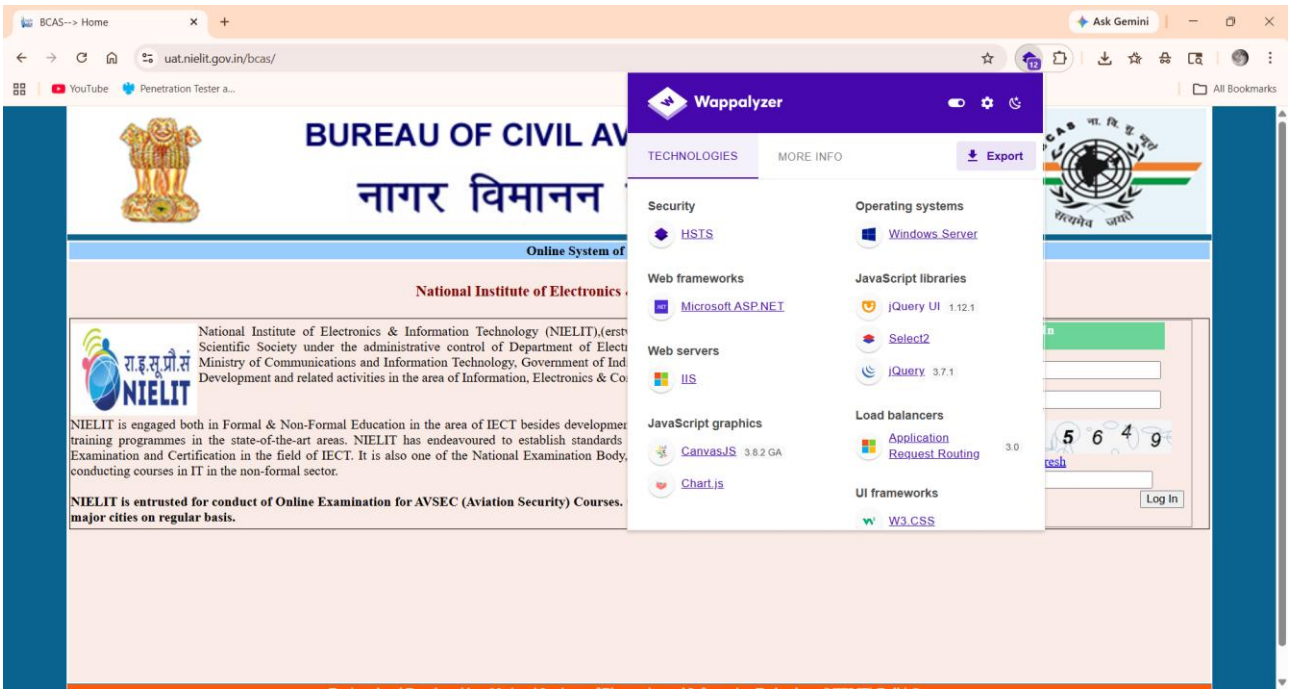
Impact:

Outdated libraries may expose the application to known vulnerabilities such as Cross-Site Scripting (XSS), increasing the risk of client-side attacks and compromise of user data.

Affected items:

- https://uat.nielit.gov.in/bcas/

Proof of Concept:



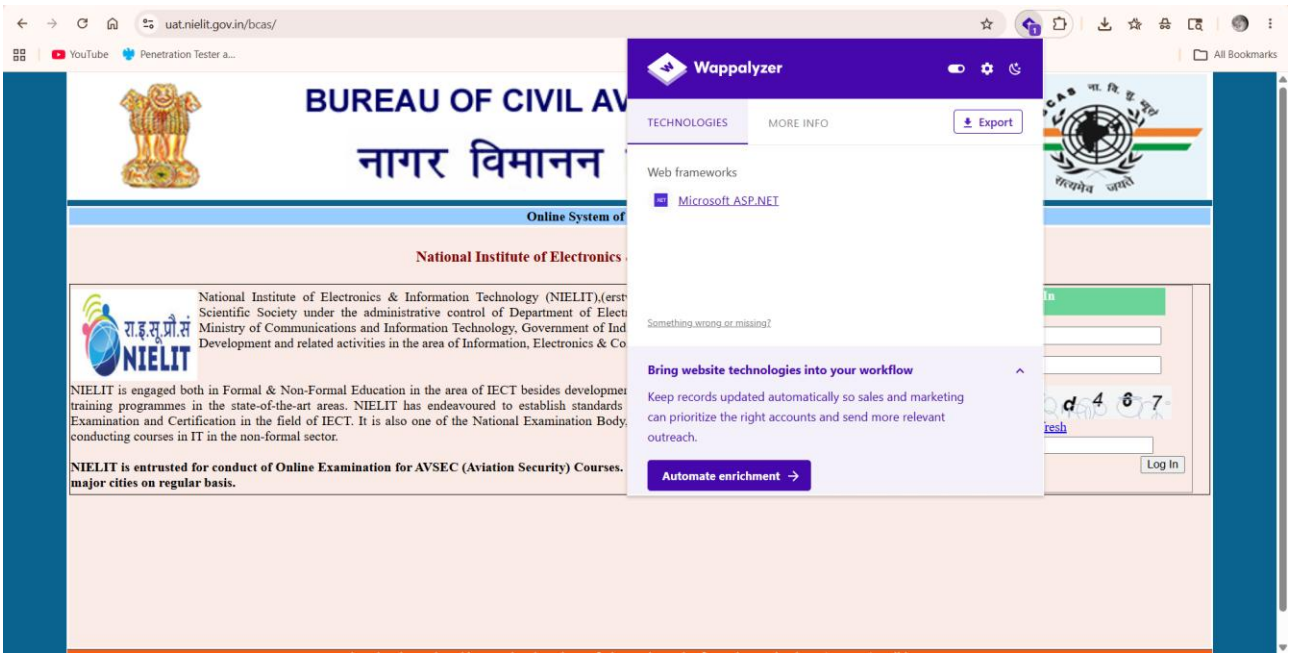
Recommendation:

- Upgrade jQuery UI to the latest stable version.
- Remove unused or deprecated libraries.
- Regularly monitor third-party components for vulnerabilities.
- Use dependency management tools to track updates.
- Perform periodic security audits of client-side libraries.

References:

https://owasp.org/www-project-top-ten/

Phase 2:



5. General Guidelines

Validate Input and Output

The input that the server receives from the user can lead to malicious code entering the server. Similarly, the output shown to the user can transmit malicious code to the client system. All user input and output should be checked to ensure it is both appropriate and expected. Input validation should be done on the client-side as well as on the server-side.

There are three main models to consider about when designing a data validation strategy.

Accept Only Known Valid Data

A character set may be defined for each field where input from the user is accepted. E.g. “A-Z, a-z, @,., 0-9, _” is a character set for a field that accepts user email.

Reject Known Bad Data

A character set of bad data may be defined for the site that has to be rejected. E.g. “CREATE, DROP, OR”

Sanitize Known Bad Data

A character set of bad data is defined and any input field that has such a character is modified. E.g. “If there is a single quote (‘) in the data, it is replaced with two single quotes.”

All methods must check:

- Data Type
- Syntax
- Length

It is recommended to use the strategy of “Accept only known data”. Further all the allowed input/output data must be sanitized on the server side by replacing scripts tags, sent as part of user input/output, with appropriate representations.

For example

- “<” by <
- “>” by >
- “(“ by &#40

This would avoid scripts from being executed on the client side.

Client-side input must also be checked for URL encoded data. URL encoding sometimes referred to as percent encoding, is the accepted method of representing characters within a URI that may need special syntax handling to be correctly interpreted. This is achieved by encoding the character to be interpreted with a sequence of three characters. This triplet sequence consists of the percentage character “%” followed by the two hexadecimal digits representing the octet code of the original character. For example, the US-ASCII character set represents a space with octet code 32, or hexadecimal 20. Thus, its URL-encoded representation is %20.

Other common characters that can be used for malicious purposes and their URL encoded representations are: -

#	Character	URL encoded
1.	‘	%27
2.	“	%22
3.	;	%3b

4.	<	%3c
5.	=	%3d
6.	>	%3e
7.	)	%29
8.	(	%28
9.	space	%20

All input validation checks should be completed after the data has been decoded and validated as acceptable content (e.g. maximum and minimum lengths, correct data type, does not contain any encoded data, textual data only contains the characters a-z and A-Z etc.)

A one-time check on the database is to be made for invalid malicious data. This would enable removal of input that has not been validated in earlier sessions. As otherwise the invalid data may cause script execution on the user's browser.

Session Management / Strong session tracking

Session Tokens on Logout

In shared computing environments, session tokens take on a new risk. If the session tokens are not invalidated after logout, they can be reused to gain access to the application. It is imperative for the application to remove the cookies from both the server and the client side after the user has logged out. The user session maintained on the server side must also be invalidated immediately after logout.

Session Time-out

All the user Session tokens must be timed-out after a certain interval of user inactivity. The Session tokens that do not expire on the HTTP server can allow an attacker unlimited time to guess or brute force a valid authenticated session token. If a user's cookie file is captured or brute-forced, then an attacker can use these static-session tokens to gain access to that user's web accounts. Additionally, session tokens can be potentially logged and cached in proxy servers that, if broken into by an attacker, may contain similar sorts of information in logs that can be exploited if the particular session has not been expired on the HTTP server.

Session Token Transmission

If a session token is captured in transit through network interception, a web application account is then trivially prone to a replay or hijacking attack. Typical web encryption technologies include Secure Sockets

Layer (SSLv2/v3) and Transport Layer Security (TLS v1) protocols in order to safeguard the state mechanism token.

Some more key points to remember:

Session ID's that are used should have the following properties:

1. Randomness.
    a. Session Ids must be randomly generated.
    b. Session Ids must be unpredictable.
    c. Make use of non-linear algorithms to generate session ID's
2. Session ID Size
    a. The size of a session ID should be large enough to ensure that it is not vulnerable to a brute force attack.
    b. The character set used should be complex. i.e. Make use of special characters.
    c. A length of 50 random characters is advised.

Salted Hashing

What is salted hashing?

The process starts with 2 elements of data:

1.) A clear text string (this could represent a password for instance).

2.) The salt, a random seed of data. This is the value used to augment a hash in order to ensure that 2 hashes of identical data yield different output.

In pseudo code we generate a salted hash as follows:

1.) Get the source string and salt as separate binary objects

2.) Concatenate the 2 binary values

3.) SHA hash the concatenation into Salted Password Hash

4.) Base64 Encode (concat (Salted Password Hash, Salt))

Credentials should be encrypted using salted hashes, so that even if the hashes are sniffed the possibility of replay attack does not exist.

References**: http://www.owasp.org/images/3/33/Salted_Hashes_Demystified.doc**

Cache Control Directives

Pages that contain sensitive information should not be stored in the local cache of the browser. To enforce this, HTTP directives need to be specified in the response. These HTTP directives need to be used to prevent enlisting of links on the browser history. The following HTTP directives can be sent by the server along with the response to the client. This would direct the browser to send a new request to the server each time it is generated.

Expires: <a previous date>, for e.g. Expires: Thu, 10 Jan 2020 19:20:00 GMT

Cache-Control: private

- ***Cache-Control: no-cache***
- ***Cache-Control: no-store***
- ***Cache-Control: must-revalidate***
- ***Pragma: no-cache***

The directive “Cache-Control: must-revalidate” directs the browser to fetch the pages from the server rather than picking it up from the local “Temporary Internet Folders”. It also directs the browser to remove the file from the temporary folders.